

Code-Layout, Readability and Reusability

Date	06 July 2026
Team ID	
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
------	-------------------------	-----------------------	--------------	--

1	Trained ML Model (hdi_model.pkl)	Python / Scikit-learn	Loaded in the Flask backend for every prediction request.	High
---	----------------------------------	-----------------------	---	------

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Adhered to standard PEP 8 spacing for all Python/Flask scripts.
2	Proper File Structure	Files and folders are logically organized	Yes	Clear separation of templates/, static/, models/, and app.py.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Used descriptive names like life_expectancy and hdi_prediction.
4	Function / Method Names	Functions are descriptively named	Yes	Clear backend routing functions such as def predict_hdi():.
5	Code Comments	Inline and block comments explain logic	Yes	Jupyter Notebook and Flask API contain explanatory docstrings.
6	Modular Design	Code is split into reusable functions/modules	Yes	Machine learning model is trained separately and imported as a .pkl file.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Cleaned up testing cells and redundant frontend scripts before deployment.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try-except blocks implemented in Flask to catch invalid numerical inputs.

Reusable Components / Modules:

2	Data Scaler (scaler.pkl)	Python / Scikit-learn	Used in the backend to normalize raw user inputs.	High
3	Prediction API Route (/predict)	Python / Flask	Can be called by any frontend web form or external service.	High
4	Main Dashboard (index.html)	HTML / CSS	Acts as the reusable base template for the user interface.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Excellent directory structure mapping frontend assets to backend logic.
Readability	5	Highly readable Python code with consistent PEP 8 formatting and clear variable naming.
Reusability	5	The decoupled architecture (ML model saved as a serialized object) maximizes reusability.
Documentation / Comments	4	Good inline commenting; API endpoints are well-documented for frontend integration.

Overall Score	4.5	Excellent Team Work and Process.
---------------	-----	----------------------------------